Maximum sub-array

Maximum sub-array

Input: An array A

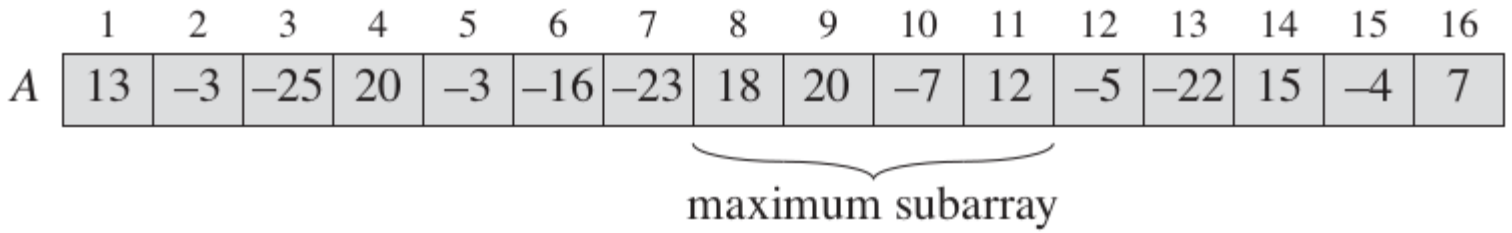
Question: Find a nonempty, contiguous subarray of A whose values have the largest sum.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
A	13	–3	–25	20	–3	–16	–23	18	20	–7	12	–5	–22	15	–4	7

Maximum sub-array

Input: An array A

Question: Find a nonempty, contiguous subarray of A whose values have the largest sum.



Maximum sub-array

Approach	Time	Space
Brute Force		
Divde and Conquer		
Dynamic programming		
Improved		

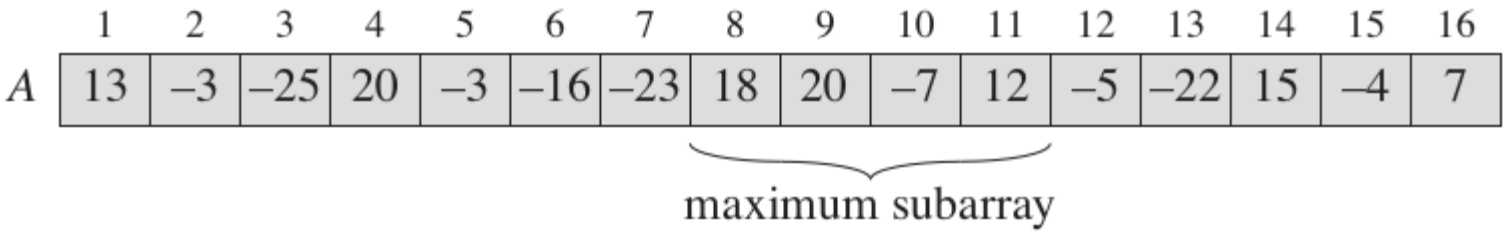
Maximum sub-array

Approach	Time	Space
Brute Force	$O(n^2)$	O(1)
Divde and Conquer		
Dynamic programming		
Improved		

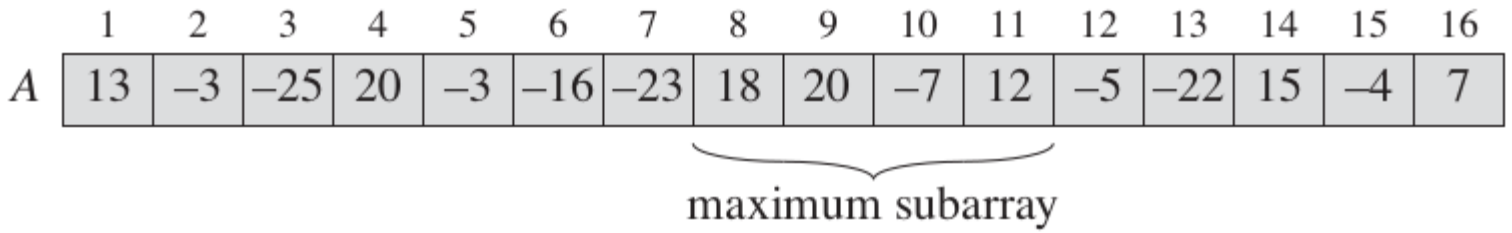
How $O(n^2)$?

Divide and Conquer

- An contiguous subarray A[i...j] of A[low, … , high] must lie exactly in one of the three cases
  - ✓ Entirely in A[low, … , mid]
  - ✓ Entirely in A[mid+1, … , high]
  - ✓ Crossing the midpoint

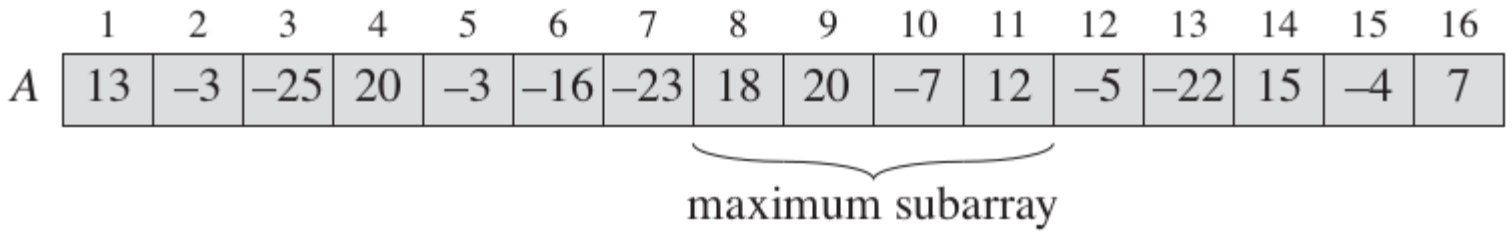


Intuition: D&C



Divide and Conquer

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

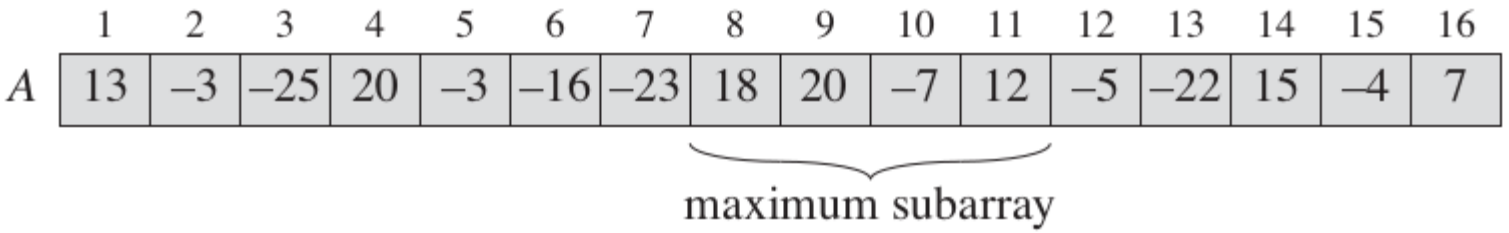


Input: 1, 5, -10, 9, -4, 6, 5, -1

Maximum sub-array

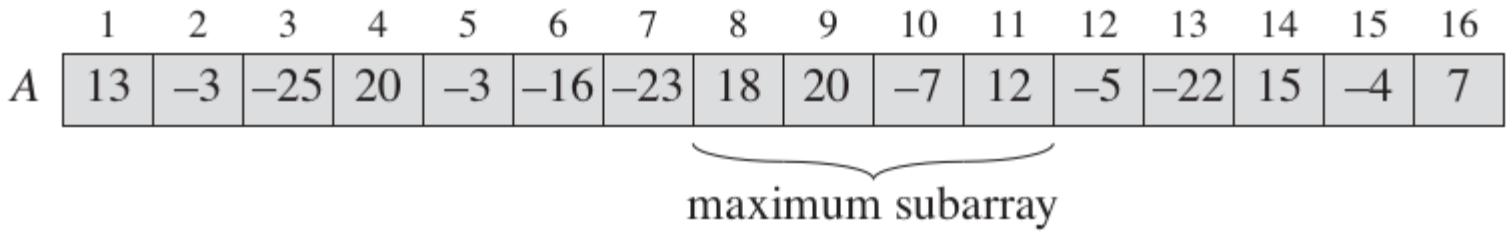
Approach	Time	Space
Brute Force	O(n^2)	O(1)
Divde and Conquer	O(n log n)	O(1)
Dynamic programming		

Intuition: DP



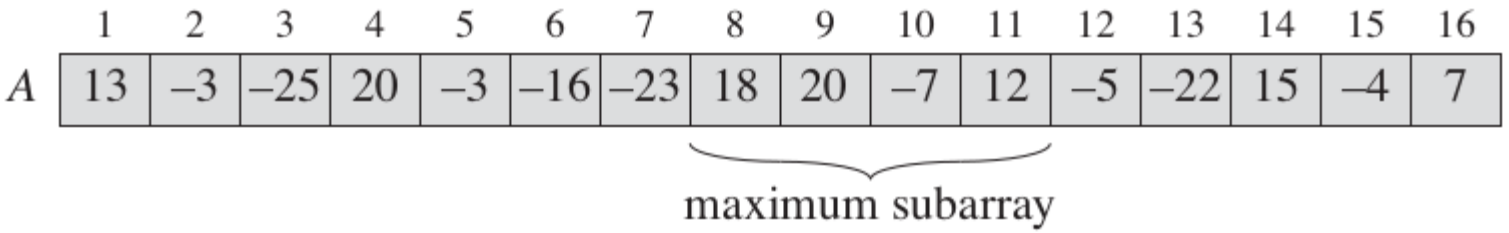
DP

- Either take an element or do not take it.
- Maintain the maximum sum till an index
  - M_i =



DP

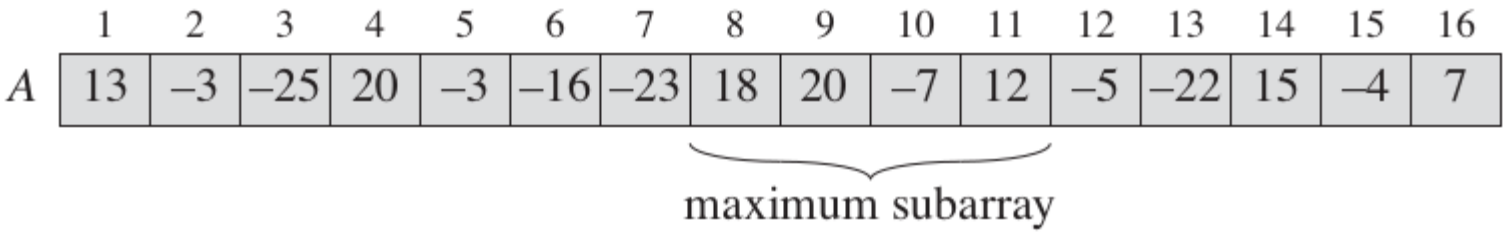
- Either take an element or do not take it.
- Maintain the maximum sum till an index
  - M_i = maximum sum of a sub-array ending **exactly** at index I



DP

- Either take an element or do not take it.
- Maintain the maximum sum till an index
  - M_i = maximum sum of a sub-array ending **exactly** at index I

$$\mathbf{M_i = max\{ M_{i-1} + A[i], A[i] \}}$$



Input: 1, 5, -10, 9, -4, 6, 5, -1

Maximum sub-array

Approach	Time	Space
Brute Force	O(n^2)	O(1)
Divde and Conquer	O(n log n)	O(1)
Dynamic programming (**Kadane's Algorithm**)	O(n)	O(n)
Improved	O(n)	O(1)

Maximum sub-array of size K

Maximum sub-array of sum K

Input: An array A, and an integer K

Question: Find a nonempty, contiguous subarray of A whose values have the sum K.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
A	13	–3	–25	20	–3	–16	–23	18	20	–7	12	–5	–22	15	–4	7

Maximum sub-array of sum K

Input: An array A, and an integer K

Question: Find a nonempty, contiguous subarray of A whose values have the sum K.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
A	13	–3	–25	20	–3	–16	–23	18	20	–7	12	–5	–22	15	–4	7

Does the D&C technique work?

Maximum sub-array of sum K

Input: An array A, and an integer K

Question: Find a nonempty, contiguous subarray of A whose values have the sum K.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
A	13	–3	–25	20	–3	–16	–23	18	20	–7	12	–5	–22	15	–4	7

What about DP?

Maximum sub-array of sum K

Input: An array A, and an integer K

Question: Find a nonempty, contiguous subarray of A whose values have the sum K.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
A	13	–3	–25	20	–3	–16	–23	18	20	–7	12	–5	–22	15	–4	7

Sliding Window algo?

Maximum sub-array of sum K

Input: An array A, and an integer K

Question: Find a nonempty, contiguous subarray of A whose values have the sum K.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
A	13	–3	–25	20	–3	–16	–23	18	20	–7	12	–5	–22	15	–4	7

Prefix sum technique

Majority element

Majority element

Input: An array A,

Question: Given an array of size n, find the element that appears more than $\lfloor n/2 \rfloor$ times.

Majority element

Input: An array A,

Question: Given an array of size n, find the element that appears more than $\lfloor n/2 \rfloor$ times.

Approach	Time	Space
Brute Force	$O(n^2)$	O(1)
Sort	O(n log n)	O(n)
Divide and Conquer		
Dynamic programming		

Majority element

Input: An array A,

Question: Given an array of size n, find the element that appears more than ⌊n/2⌋ times.

Approach	Time	Space
Brute Force	O(n^2)	O(1)
Sort	O(n log n)	O(n)
Divide and Conquer	O(n log n)	O(1)
Dynamic programming	??	??
Boyer-Moore voting algorithm	O(n)	O(1)

Majority element

Input: 10, 15, 10, 15, 10, 15, 10

1) Initialize two variables:
   - count = 0 (to track balance)
   - candidate = None (to hold the current candidate for majority)
2) Traverse the array:
   - If count == 0, choose current element as candidate
   - If current element equals candidate, increment count
   - Else, decrement count
3) After one pass, candidate will hold the majority element (if it exists)

Peak element

Peak element

Input: An array A of distinct elements,

Question: Find a peak element A[i] of A, where

A[i] > A[i+1] and A[i] <A[i-1].

If i=1 then A[1] > A[2] and

If i=n then A[n]>A[n-1].

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
A	13	–3	–25	20	–3	–16	–23	18	20	–7	12	–5	–22	15	–4	7

Peak element

Q: Does there exists a peak element?

Input: An array A of distinct elements,

Question: Find a peak element A[i] of A, where

A[i] > A[i+1] and A[i] <A[i-1].

If i=1 then A[1] > A[2] and

If i=n then A[n]>A[n-1].

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
A	13	–3	–25	20	–3	–16	–23	18	20	–7	12	–5	–22	15	–4	7

Peak element

Approach	Time	Space
Brute Force	O(n)	O(1)
Divide and Conquer	O(log n)	
Dynamic programming	??	??

Peak element

Approach	Time	Space
Brute Force	O(n)	O(1)
Divide and Conquer	O(log n)	
Dynamic programming	O(n)	O(n)